You

In python, I have teh following method where an example of source_path is "Example filled Breaking Cycles Referral Form 3.docx" and destination_path is "Other AI Projects/AI Document Extraction/src/localFiles/Example filled Breaking Cycles Referral Form 3.docx":

```
def upload_file(self, source_path, destination_path):
  # Copy the file to the localFiles directory
  try:
    shutil.copy(source_path, destination_path)
    print(f"File successfully copied to {destination_path}")
  except FileNotFoundError:
    print("The source file does not exist. Please check the path.")
  except Exception as e:
    print(f"An error occurred: {e}")
```

How can I make a similar method where the file is actually uploaded into teh local directory so that it tehn has destination path, given that it will always be a word file (maybe using docx?)?

Copilot

To create a method that uploads a Word file to a local directory and ensures the destination path is properly set, you can use the python-docx library to handle Word files. This approach allows you to read the contents of the Word file, create a new file at the destination path, and write the contents to this new file.

Here's how you can modify your method:

```
import shutil
from docx import Document

def upload_file(source_path, destination_path):
    # Copy the file to the localFiles directory
    try:
        # Read the source Word file
        doc = Document(source_path)

        # Save the document to the destination path
        doc.save(destination_path)

        print(f"File successfully uploaded to {destination_path}")
    except FileNotFoundError:
        print("The source file does not exist. Please check the path.")
    except Exception as e:
        print(f"An error occurred: {e}")
```

In this updated method:

1. The Document object from the docx library is used to read the source Word file.
2. The doc.save(destination_path) method saves the document to the specified destination path.

This ensures that the Word file is properly handled as a document, making it easier to manipulate or extract content from it if needed. Before running this code, ensure you have the python-docx library installed, which you can do by running:

pip install python-docx

Let me know if there's anything more specific you're working on with these files!

You

Ok, how can I extract data from one document based on teh following attributes and methods I have that are part of the documentExtractor class, and I want to populate another document with teh result according to the mappings, how can I refine this process?
Current relevant code:

```
#Initialising an attribute to store the name of the input file that is being extracted from
self.input_file = ""

#Storing mappings of numbers to the relevant sections in the order of the form
self.section_mappings = {
  1: "Main Title",
  2: "Sub Title",
  3: "Name",
  4: "Info",
  5: "DoB & Gender",
  6: "Tick boxes",
  7: "Address",
  8: "Tick boxes",
  9: "Phone num",
  10: "Tick boxes",
  11: "Refer Info",
  12: "Tick boxes",
  13: "Date and School",
  14: "Tick boxes",
  15: "Safety Concerns",
  16: "Tick boxes",
  17: "Main Reasons Heading",
  18: "Case History Heading",
  19: "Main Reasons Info",
  20: "Case History Info",
  21: "Helpful Work Heading",
  22: "Please Tick Heading",
  23: "Notes Heading",
  24: "Partner access support and info",
  25: "Partner access support and info tickbox",
  26: "Partner access support and info notes",
  27: "One to one intervention",
  28: "One to one intervention tickbox",
  29: "One to one intervention notes",
  30: "One to one support",
  31: "One to one support tickbox",
  32: "One to one support notes",
  33: "Criminality reduction",
  34: "Criminality reduction tickbox",
  35: "Criminality reduction notes",
  36: "Outcome Support",
  37: "Outcome Support tickbox",
  38: "Outcome Support notes",
  39: "Other work",
  40: "Other work tickbox",
  41: "Other work notes",
  42: "Referral Date Heading",
  43: "Referral Date Info",
```

```
  44: "Allocation Date Heading",
  45: "Allocation Date Info",
  46: "Closure Date Heading",
  47: "Closure Date Info"
}
# Mapping dictionary between the 2 forms
self.mapping = {
  "Name (Young Person):": "YOUNG PERSON DETAILS",
  "Date of Birth (Young Person):": "D.O.B / AGE",
  "Gender:": "YOUNG PERSON DETAILS",
  "Address (Young Person):": "YOUNG PERSON DETAILS",
  "Postcode:": "YOUNG PERSON DETAILS",
  "Phone Number (Young Person):": "YOUNG PERSON DETAILS",
  "Name of person making referral:": "REFERRAL DETAILS NAME / CONTACT No",
  "Address of person / organisation making referral:": "REFERRAL DETAILS NAME / CONTACT No",
  "Email of person making referral:": "REFERRAL DETAILS NAME / CONTACT No",
  "Telephone number of person making referral:": "REFERRAL DETAILS NAME / CONTACT No",
  "Date of referral:": "Date Case Allocated",
  "School attended:": "SCHOOL / COLLEGE ATTENDANCE %",
}

# Function to extract text from a Word document
def extract_text_from_docx(self, file_path):
  doc = Document(file_path)
  full_text = []
  section_num = 1
  #with title {self.section_mappings[section_num]}
  for table in doc.tables:
    for row in table.rows:
      for cell in row.cells:
        full_text.append(f"This is section {str(section_num)}")
        full_text.append(cell.text)
        section_num += 1
  return "\n".join(full_text)

# Function to map extracted data to new labels
def map_data(self, extracted_data):
  mapped_data = {}
  new_data = ""
  for old_label, new_label in self.mapping.items():
    if old_label in extracted_data:
      try:
        print("Old mapped data:",mapped_data[old_label])
      except KeyError:
        pass
      try:
        print("New extracted data:",extracted_data[new_label])
      except KeyError:
        pass
      print("Old extracted data:",extracted_data[old_label])
      if extracted_data[old_label] in mapped_data.values():
        mapped_data[new]
      mapped_data[new_label] = extracted_data[old_label]
      print("New mapped data:",mapped_data[new_label])
  return mapped_data

# Function to extract specific fields using regex and keyword matching
def custom_field_extraction(self, text):
  extracted_data = {}

  # Extract name (Example: Look for patterns like capitalized words, typically names)
  name_match = re.search(r"Name \(Young Person\):\n*(.*)", text)
  if name_match:
    extracted_data['Name (Young Person):'] = name_match.group(1).strip()
```

```
  # Extract Date of Birth
  dob_match = re.search(r"Date of Birth \(Young Person\):\s*(\d{2} \w+ \d{4})", text)
  if dob_match:
    extracted_data['Date of Birth (Young Person):'] = dob_match.group(1).strip()

  # Extract Address
  address_match = re.search(r"Address \(Young Person\):\n*(.*)", text)
  if address_match:
    extracted_data['Address (Young Person):'] = address_match.group(1).strip()

  # Extract Phone Number
  phone_match = re.search(r"Phone Number \(Young Person\):\n*(\d+)", text)
  if phone_match:
    extracted_data['Phone Number (Young Person):'] = phone_match.group(1).strip()

  # Extract other fields similarly
  # ...

  return extracted_data

def extractInfo(self, file_path):
  # Extract text from the document
  uploaded_text = self.extract_text_from_docx(file_path)
  print(uploaded_text)

  # Perform custom field extraction on the extracted text
  extracted_data = self.custom_field_extraction(uploaded_text)
  print("Extracted Data:", extracted_data)

  # Map the extracted data
  mapped_data = self.map_data(extracted_data)
  print("Mapped Data:", mapped_data)

  #Set the internally held file path now
  self.input_file = file_path

  #Send the mapped data back to the main part of the program
  return mapped_data
```

Copilot

You have a solid foundation, and it looks like you're making good progress! Let's refine the process to extract data from one document and populate another according to the mappings. Here's an updated version of your code with a method to create a new document using the extracted and mapped data:

1. Extract data from the source document.
2. Map the extracted data to new labels.
3. Create a new document and populate it with the mapped data.

Here's the enhanced version of your documentExtractor class:

```
import re
from docx import Document

class documentExtractor:
    def __init__(self):
        self.input_file = ""
        self.section_mappings = { ... }  # Your existing section mappings
        self.mapping = { ... }  # Your existing mapping dictionary

    def extract_text_from_docx(self, file_path):
```

```
        doc = Document(file_path)
        full_text = []
        section_num = 1
        for table in doc.tables:
            for row in table.rows:
                for cell in row.cells:
                    full_text.append(f"This is section {str(section_num)}")
                    full_text.append(cell.text)
                    section_num += 1
        return "\n".join(full_text)

    def custom_field_extraction(self, text):
        extracted_data = {}
        name_match = re.search(r"Name \(Young Person\):\n*(.*)", text)
        if name_match:
            extracted_data['Name (Young Person):'] = name_match.group(1).strip()
        dob_match = re.search(r"Date of Birth \(Young Person\):\s*(\d{2} \w+ \d{4})", text)
        if dob_match:
            extracted_data['Date of Birth (Young Person):'] = dob_match.group(1).strip()
        address_match = re.search(r"Address \(Young Person\):\n*(.*)", text)
        if address_match:
            extracted_data['Address (Young Person):'] = address_match.group(1).strip()
        phone_match = re.search(r"Phone Number \(Young Person\):\n*(\d+)", text)
        if phone_match:
            extracted_data['Phone Number (Young Person):'] = phone_match.group(1).strip()
        # Extract other fields similarly...
        return extracted_data

    def map_data(self, extracted_data):
        mapped_data = {}
        for old_label, new_label in self.mapping.items():
            if old_label in extracted_data:
                mapped_data[new_label] = extracted_data[old_label]
        return mapped_data

    def extract_info(self, file_path):
        uploaded_text = self.extract_text_from_docx(file_path)
        extracted_data = self.custom_field_extraction(uploaded_text)
        mapped_data = self.map_data(extracted_data)
        self.input_file = file_path
        return mapped_data

    def create_new_document(self, destination_path, mapped_data):
        new_doc = Document()
        for section, content in mapped_data.items():
            new_doc.add_heading(section, level=1)
            new_doc.add_paragraph(content)
        new_doc.save(destination_path)
        print(f"New document created at {destination_path}")

# Example usage:
extractor = documentExtractor()
mapped_data = extractor.extract_info("source_path.docx")
extractor.create_new_document("destination_path.docx", mapped_data)
```

In this refined process:

1. extract_info: Extracts and maps data from the source document.
2. create_new_document: Creates a new document based on the mapped data and saves it to the specified destination path.

Feel free to adapt and expand on this structure based on your specific requirements. Let me know if there's anything else I can help you with!

You

(Now Wednesday, Nov 20)
For the map_data function I want to make it so that if new_label is already in the mapped_data then it adds on the new extracted_data to the current item in mapped_data

Copilot

Sure, we can modify the map_data function to append new extracted data to the existing item if the new_label is already in mapped_data. Here's how you can do it:

```
def map_data(self, extracted_data):
    mapped_data = {}
    for old_label, new_label in self.mapping.items():
        if old_label in extracted_data:
            if new_label in mapped_data:
                # Append new extracted data to the existing item
                mapped_data[new_label] += " " + extracted_data[old_label]
            else:
                # Add new item to mapped_data
                mapped_data[new_label] = extracted_data[old_label]
    return mapped_data
```

In this updated function:

- We first check if old_label is present in extracted_data.
- Then, we check if new_label already exists in mapped_data.
    - If it does, we append the new extracted data to the existing item.
    - If it doesn't, we add a new item to mapped_data.

This ensures that any additional data for an already existing label is appended to it, rather than overwriting it. Give it a try and let me know how it works for you!

You

Can you complete the custom_field_extraction using this format for the rest of the items in the mappings dictionary?

```
# Extract name (Example: Look for patterns like capitalized words, typically names)
name_match = re.search(r"Name \(Young Person\):\n*(.*)", text)
if name_match:
  extracted_data['Name (Young Person):'] = name_match.group(1).strip()

# Extract Date of Birth
dob_match = re.search(r"Date of Birth \(Young Person\):\s*(\d{2} \w+ \d{4})", text)
if dob_match:
  extracted_data['Date of Birth (Young Person):'] = dob_match.group(1).strip()

# Extract other fields similarly
# ...

self.mapping = {
  "Name (Young Person):": "YOUNG PERSON DETAILS",
  "Date of Birth (Young Person):": "D.O.B / AGE",
  "Gender:": "YOUNG PERSON DETAILS",
  "Address (Young Person):": "YOUNG PERSON DETAILS",
  "Postcode:": "YOUNG PERSON DETAILS",
  "Phone Number (Young Person):": "YOUNG PERSON DETAILS",
  "Name of person making referral:": "REFERRAL DETAILS NAME / CONTACT No",
  "Address of person / organisation making referral:": "REFERRAL DETAILS NAME / CONTACT No",
  "Email of person making referral:": "REFERRAL DETAILS NAME / CONTACT No",
  "Telephone number of person making referral:": "REFERRAL DETAILS NAME / CONTACT No",
  "Date of referral:": "Date Case Allocated",
  "School attended:": "SCHOOL / COLLEGE ATTENDANCE %",
}
```

Copilot

Absolutely, I can help with that. Here's the complete custom_field_extraction method using regex to extract all the fields listed in your mappings dictionary:

```
def custom_field_extraction(self, text):
    extracted_data = {}

    # Extract Name
    name_match = re.search(r"Name \(Young Person\):\n*(.*)", text)
    if name_match:
        extracted_data['Name (Young Person):'] = name_match.group(1).strip()

    # Extract Date of Birth
    dob_match = re.search(r"Date of Birth \(Young Person\):\s*(\d{2} \w+ \d{4})", text)
    if dob_match:
        extracted_data['Date of Birth (Young Person):'] = dob_match.group(1).strip()

    # Extract Gender
    gender_match = re.search(r"Gender:\s*(.*)", text)
    if gender_match:
        extracted_data['Gender:'] = gender_match.group(1).strip()

    # Extract Address
    address_match = re.search(r"Address \(Young Person\):\n*(.*)", text)
    if address_match:
        extracted_data['Address (Young Person):'] = address_match.group(1).strip()

    # Extract Postcode
    postcode_match = re.search(r"Postcode:\s*(.*)", text)
    if postcode_match:
        extracted_data['Postcode:'] = postcode_match.group(1).strip()

    # Extract Phone Number
    phone_match = re.search(r"Phone Number \(Young Person\):\n*(\d+)", text)
    if phone_match:
        extracted_data['Phone Number (Young Person):'] = phone_match.group(1).strip()

    # Extract Name of person making referral
    ref_name_match = re.search(r"Name of person making referral:\s*(.*)", text)
    if ref_name_match:
        extracted_data['Name of person making referral:'] = ref_name_match.group(1).strip()

    # Extract Address of person/organisation making referral
    ref_address_match = re.search(r"Address of person / organisation making referral:\n*(.*)", text)
    if ref_address_match:
        extracted_data['Address of person / organisation making referral:'] = ref_address_match.group(1).strip()

    # Extract Email of person making referral
    ref_email_match = re.search(r"Email of person making referral:\s*(.*)", text)
    if ref_email_match:
        extracted_data['Email of person making referral:'] = ref_email_match.group(1).strip()

    # Extract Telephone number of person making referral
    ref_phone_match = re.search(r"Telephone number of person making referral:\s*(\d+)", text)
    if ref_phone_match:
        extracted_data['Telephone number of person making referral:'] = ref_phone_match.group(1).strip()

    # Extract Date of referral
    ref_date_match = re.search(r"Date of referral:\s*(.*)", text)
    if ref_date_match:
        extracted_data['Date of referral:'] = ref_date_match.group(1).strip()

    # Extract School attended
    school_match = re.search(r"School attended:\s*(.*)", text)
    if school_match:
        extracted_data['School attended:'] = school_match.group(1).strip()

    return extracted_data
```

This method uses regular expressions to search for the specified patterns in the text and extract the corresponding values. The extracted data is then stored in the extracted_data dictionary with the keys matching the labels in your mapping dictionary.

Now your extraction method should cover all the fields according to your mapping dictionary. If you need any more adjustments or additional fields, just let me know!

Conversation by Fifa Utuber with Copilot on Monday, Nov 18 2024